

Iterator e Visitor

Padrões de Projeto - Aula #12

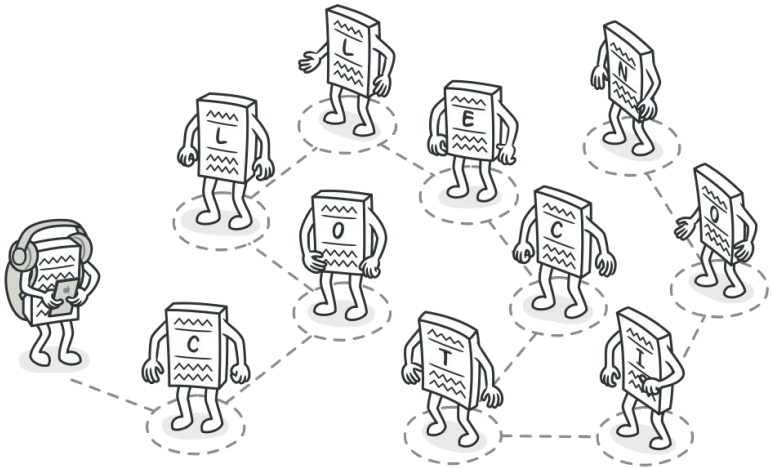
Prof. Dr. Carlos Melo

carlos.melo@upe.br

UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Padrões Comportamentais (parte #5)

Iterator
Percorrer uma coleção sem expor sua representação interna



Visitor
Separar operações dos objetos sobre os quais elas são executadas



Objetivos de aprendizagem

- Distinguir a responsabilidade da Coleção da responsabilidade do Iterador
- Implementar a iteração em Python utilizando __iter__(), __next__() e StopIteration
- Criar percursos independentes sobre uma mesma coleção
- Implementar visitantes que realizam operações diferentes sobre os mesmos objetos
- Identificar quando utilizar cada padrão e avaliar seus custos

Iterator - O Problema

- Considere uma coleção que recebeu estas palavras, nesta sequência:
  - `["banana", "abacaxi", "caju"]`
- Queremos oferecer dois percursos:
  - Ordem alfabética
    - `"abacaxi", "banana", "caju"`
  - Ordem alfabética inversa
    - `"caju", "banana", "abacaxi"`
- Também queremos permitir que dois percursos estejam em andamento ao mesmo tempo, cada um com sua própria posição
- Se o Cliente precisar conhecer o armazenamento e controlar todos esses detalhes, ficará acoplado à implementação da coleção
- O Iterator separa essas responsabilidades

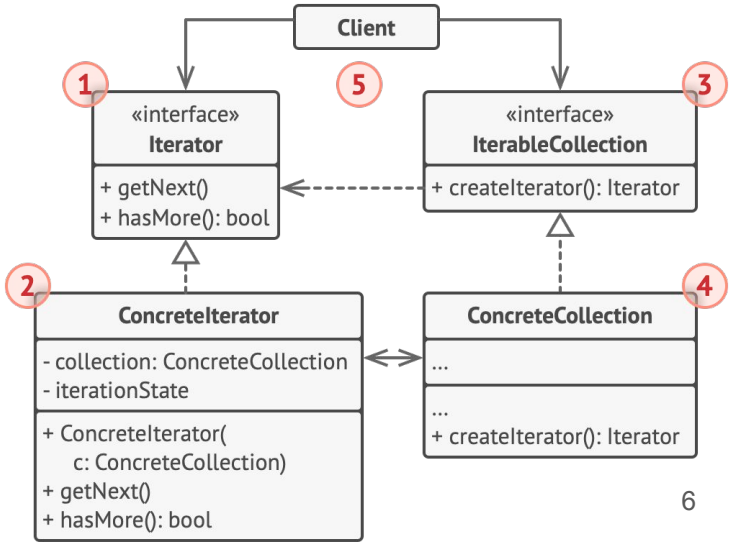
Iterator - ideia e intenção

"Permitir percorrer os elementos de uma coleção sem expor sua representação interna"

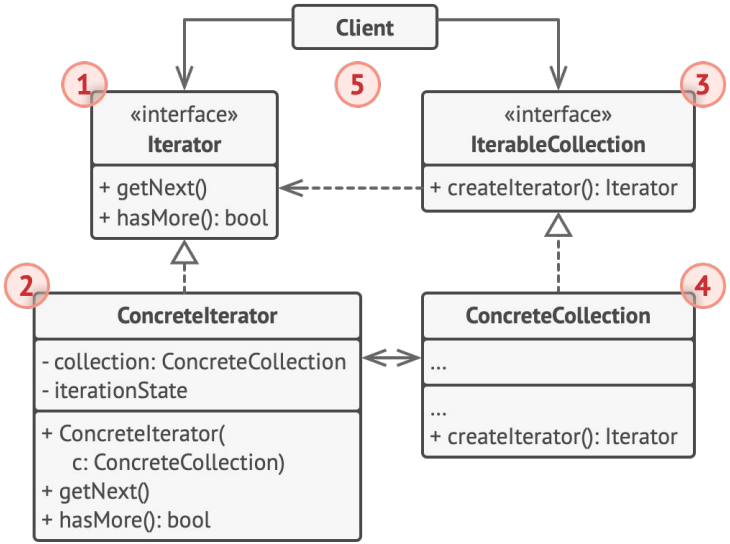
- Extrair o comportamento de percurso para um objeto separado: o Iterador
  - A Coleção armazena os elementos e fornece os iteradores
  - Cada Iterador controla sua posição e determina o próximo elemento
  - O Cliente solicita os elementos pela interface do Iterador
- Aplicando ao nosso contexto:
  - Um iterador percorre as palavras em ordem alfabética e outro realiza o percurso inverso
  - Cada iterador mantém seu próprio progresso, sem alterar a ordem de armazenamento das palavras

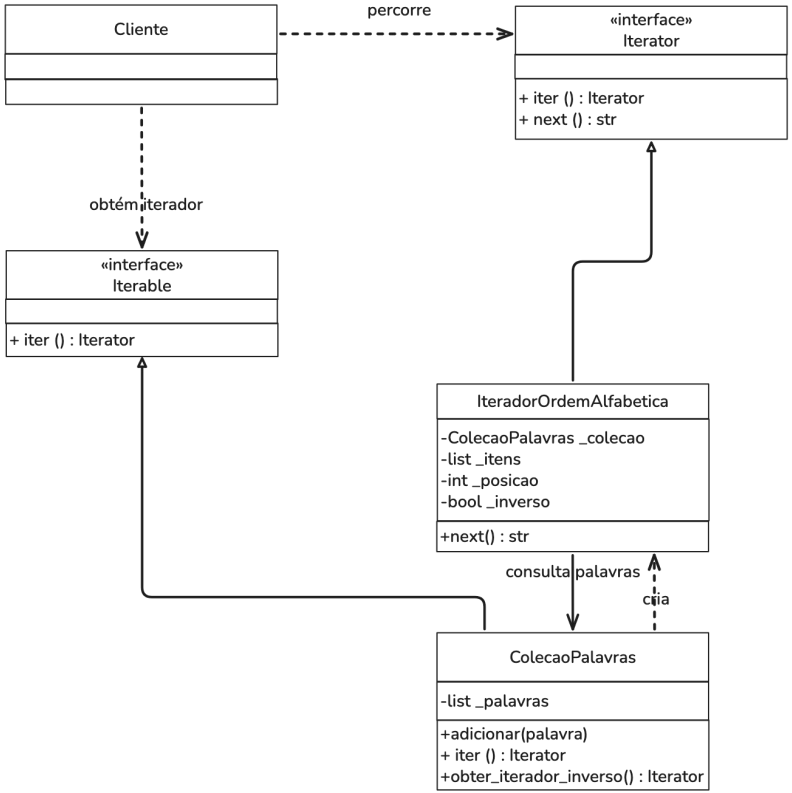
Iterator - Estrutura

1. A interface Iterador declara as operações necessárias para percorrer uma coleção
2. Iteradores Concretos implementam algoritmos específicos para percorrer uma coleção
3. A interface Coleção declara um ou mais métodos para obter os iteradores compatíveis com a coleção
4. Coleções Concretas retornam novas instâncias de uma classe iterador concreta em particular cada vez que o cliente pede por uma
5. O Cliente trabalha tanto com as coleções como os iteradores através de suas interfaces



Aplicando ao nosso contexto





Aplicando ao nosso contexto - Iterador Concreto

```python
from __future__ import annotations
from collections.abc import Iterable, Iterator


class IteradorOrdemAlfabetica(Iterator[str]):
    def __init__(
        self, colecao: ColecaoPalavras, inverso: bool = False
    ) -> None:
        self._colecao = colecao
        self._inverso = inverso
        self._itens: list[str] | None = None
        self._posicao = 0

    def __next__(self) -> str:
        if self._itens is None:
            self._itens = sorted(self._colecao._palavras, reverse=self._inverso)

        if self._posicao >= len(self._itens):
            raise StopIteration

        palavra = self._itens[self._posicao]
        self._posicao += 1
        return palavra
```

- A ordenação acontece na primeira solicitação de um elemento
- `sorted()` cria uma lista ordenada, preservando a ordem de armazenamento
- Cada instância mantém sua própria `_posicao`

Aplicando ao nosso contexto - Coleção Concreta

```python
class ColecaoPalavras(Iterable[str]):
    def __init__(self) -> None:
        self._palavras: list[str] = []

    def adicionar(self, palavra: str) -> None:
        self._palavras.append(palavra)

    def __iter__(self) -> Iterator[str]:
        return IteradorOrdemAlfabetica(self)

    def obter_iterador_inverso(self) -> Iterator[str]:
        return IteradorOrdemAlfabetica(self, inverso=True)
```

- `__iter__()` fornece o percurso alfabético
- `obter_iterador_inverso()` fornece o percurso alfabético inverso
- Cada chamada cria um novo iterador, com seu próprio progresso

Aplicando ao nosso contexto - Execução

```python
class Rascunho(Estado):
    def publicar(self, perfil: str) -> None:
        self._contexto.mudar_estado(Moderacao(self._contexto))


class Moderacao(Estado):
    def publicar(self, perfil: str) -> None:
        if perfil == "administrador":
            self._contexto.mudar_estado(Publicado(self._contexto))


class Publicado(Estado):
    def publicar(self, perfil: str) -> None:
        pass
```

- `Rascunho`: encaminha o documento para moderação
- `Moderação`: permite a publicação quando o perfil é administrador
- `Publicado`: mantém o documento publicado
- Ao criar o próximo estado, passamos a referência para o mesmo documento

Aplicando ao nosso contexto - Dois iteradores

```
primeiro = iter(colecao)
segundo = iter(colecao)

print(next(primeiro))
print(next(primeiro))
print(next(segundo))

print(list(primeiro))
print(list(segundo))
```

- Avançar primeiro não modifica a posição de segundo
- `list(iterador)` consome os elementos restantes daquele percurso
- Cada iterador pode ser utilizado em momentos diferentes

Prós e contras do Iterator

Prós

- Separa o armazenamento dos dados da lógica de percurso
- Permite intercalar percursos independentes sobre a mesma coleção
- Oferece uma forma comum de consumir coleções diferentes

Contras

- Pode acrescentar estrutura desnecessária para coleções simples
- Pode apresentar custos adicionais em relação ao acesso direto aos elementos
- Exige implementar corretamente o controle e o encerramento do percurso

Use:

- Quando precisar percorrer uma estrutura complexa sem expor seus detalhes internos ao Cliente
- Quando vários trechos da aplicação repetirem a lógica de percurso
- Quando o mesmo Cliente precisar percorrer coleções diferentes por uma interface comum

Visitor - O Problema

- Uma aplicação trabalha com cidades, indústrias e pontos turísticos
- Precisamos exportar suas informações para XML e, posteriormente, para outros formatos
  - `Cidade` exporta nome e população
  - `Industria` exporta nome e setor
  - `PontoTuristico` exporta nome e categoria
- Adicionar métodos como `exportar_xml()` e `exportar_json()` em cada classe espalharia a lógica de exportação pelas classes do domínio

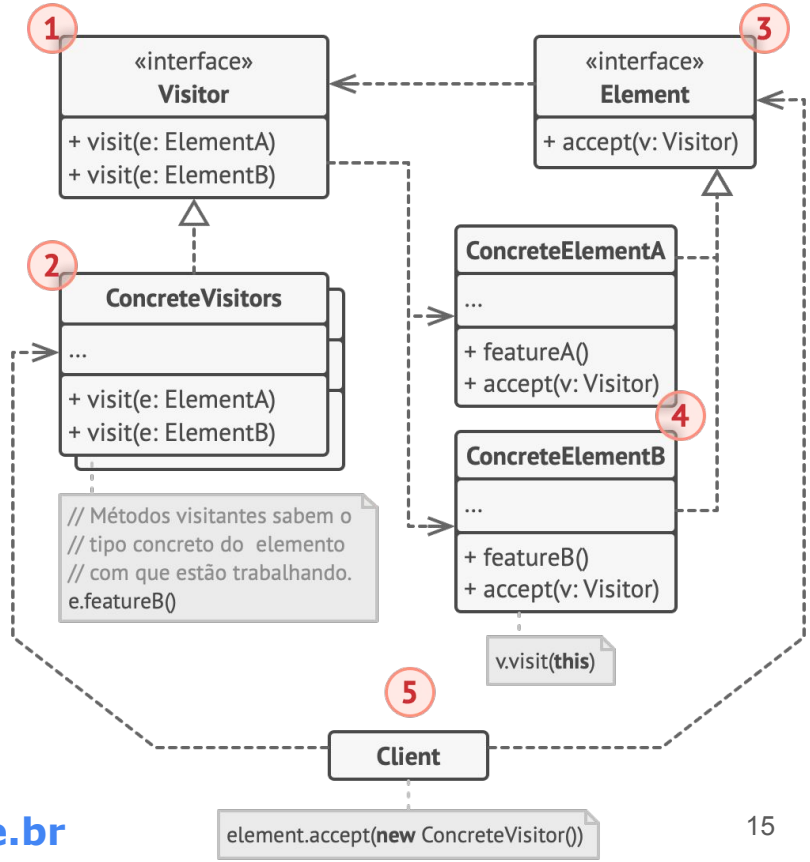
Visitor - ideia e intenção

"Separar os algoritmos dos objetos sobre os quais eles operam"

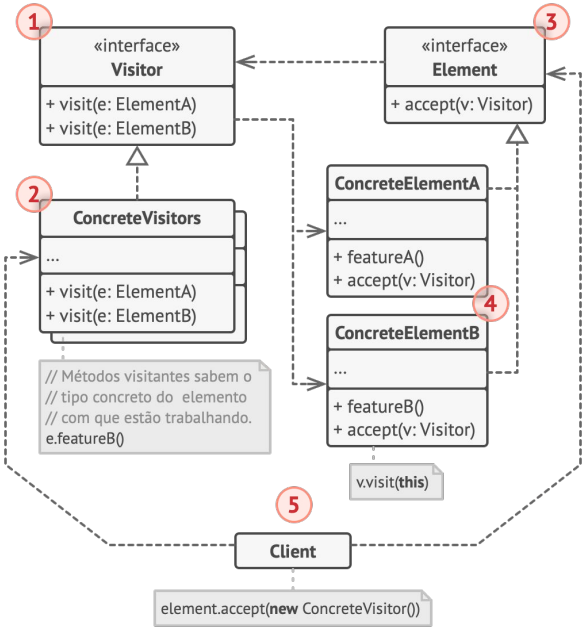
- Ideia central:
  - O Visitante reúne uma operação com tratamentos específicos para diferentes tipos de elementos
  - Cada Elemento aceita um visitante
  - O Elemento Concreto encaminha a chamada ao método correspondente ao seu tipo
- No nosso contexto `ExportadorXML` e `ExportadorJSON` processam os mesmos objetos, produzindo representações diferentes

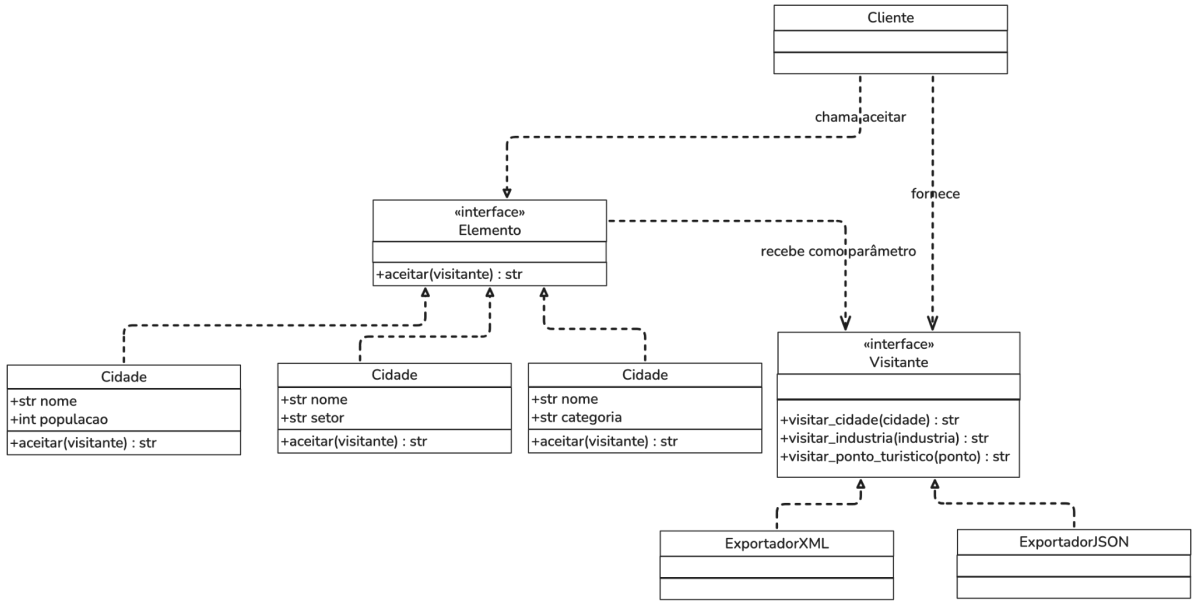
Visitor - Estrutura

1. A interface Visitante declara um conjunto de métodos visitantes que podem receber elementos concretos de uma estrutura de objetos como argumentos
2. Cada Visitante Concreto implementa diversas versões do mesmo comportamento, feitos sob medida para diferentes elementos concretos de classes
3. A interface Elemento declara um método para "aceitar" visitantes
4. Cada Elemento Concreto deve implementar o método de aceitação. O propósito desse método é redirecionar a chamada para o método visitante apropriado que corresponde com a atual classe elemento
5. O Cliente geralmente representa uma coleção de outros objetos complexos (por exemplo, uma árvore Composite)



Aplicando ao nosso contexto





Aplicando ao nosso contexto - Interfaces

```python
from __future__ import annotations
from abc import ABC, abstractmethod


class Elemento(ABC):
    @abstractmethod
    def aceitar(self, visitante: Visitante) -> str:
        pass


class Visitante(ABC):
    @abstractmethod
    def visitar_cidade(self, cidade: Cidade) -> str:
        pass

    @abstractmethod
    def visitar_industria(self, industria: Industria) -> str:
        pass

    @abstractmethod
    def visitar_ponto_turistico(self, ponto: PontoTuristico) -> str:
        pass
```

- Elemento: exige o método de aceitação
- Visitante: declara um método para cada tipo de Elemento Concreto

Aplicando ao nosso contexto - Elementos concretos

```python
from dataclasses import dataclass


@dataclass
class Cidade(Elemento):
    nome: str
    populacao: int

    def aceitar(self, visitante: Visitante) -> str:
        return visitante.visitar_cidade(self)


@dataclass
class Industria(Elemento):
    nome: str
    setor: str

    def aceitar(self, visitante: Visitante) -> str:
        return visitante.visitar_industria(self)


@dataclass
class PontoTuristico(Elemento):
    nome: str
    categoria: str

    def aceitar(self, visitante: Visitante) -> str:
        return visitante.visitar_ponto_turistico(self)
```

- Cada classe encaminha a chamada ao método correspondente do visitante
- A lógica de exportação ficará nos Visitantes Concretos

Aplicando ao nosso contexto - ExportadorXML

```python
from xml.etree.ElementTree import Element, tostring


class ExportadorXML(Visitante):
    def visitar_cidade(self, cidade: Cidade) -> str:
        no = Element("cidade", nome=cidade.nome, populacao=str(cidade.populacao))
        return tostring(no, encoding="unicode")

    def visitar_industria(self, industria: Industria) -> str:
        no = Element("industria", nome=industria.nome, setor=industria.setor)
        return tostring(no, encoding="unicode")

    def visitar_ponto_turistico(self, ponto: PontoTuristico) -> str:
        no = Element("ponto_turistico", nome=ponto.nome, categoria=ponto.categoria)
        return tostring(no, encoding="unicode")
```

- Element monta o nó `XML`, e `tostring()` produz sua representação textual
- Cada método utiliza os campos específicos do elemento recebido

Aplicando ao nosso contexto - ExportadorJSON

```python
import json


class ExportadorJSON(Visitante):
    def visitar_cidade(self, cidade: Cidade) -> str:
        return json.dumps({
          "tipo": "cidade",
          "nome": cidade.nome,
          "populacao": cidade.populacao
        }, ensure_ascii=False)

    def visitar_industria(self, industria: Industria) -> str:
        return json.dumps({
          "tipo": "industria",
          "nome": industria.nome,
          "setor": industria.setor}, ensure_ascii=False
        )

    def visitar_ponto_turistico(self, ponto: PontoTuristico) -> str:
        return json.dumps({
          "tipo": "ponto_turistico",
          "nome": ponto.nome, "categoria": ponto.categoria}, ensure_ascii=False,
        )
```

- `json.dumps()` transforma o dicionário em texto JSON
- O novo formato foi implementado sem alterar `Cidade`, `Industria` ou `PontoTuristico`

Aplicando ao nosso contexto - Execução

```python
elementos: list[Elemento] = [
    Cidade("Vila Nova", 30000),
    Industria("Fábrica Alfa", "alimentos"),
    PontoTuristico("Mirante Central", "paisagem"),
]

for visitante in (ExportadorXML(), ExportadorJSON()):
    print(type(visitante).__name__)
    for elemento in elementos:
        print(elemento.aceitar(visitante))
```

Prós e Contras do Visitor

Prós

- Acrescenta operações por meio de novos Visitantes Concretos
- Reúne os tratamentos de uma operação em uma classe
- Pode acumular resultados durante as visitas

Contras

- Acrescentar Elementos Concretos exige atualizar os visitantes
- Os visitantes precisam conhecer os tipos concretos dos elementos
- Pode precisar de dados que os elementos mantêm encapsulados

Use

- Quando uma operação precisa tratar objetos de diferentes tipos
- Quando comportamentos auxiliares, como exportação, devem ficar separados das classes do domínio
- Quando novas operações precisam ser acrescentadas com frequência.



Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS